

TP1 : Robot de cuisine

Compétences visées :

- Mise en pratique des connaissances acquises en modélisation UML :
 - diagramme de classes,
 - diagramme d'états-transitions.
- Mise en pratique des connaissances acquises en ingénierie logicielle :
 - automatisation des tâches avec Apache Maven,
 - analyse statique du code avec SonarQube,
 - journalisation d'événements avec Apache Log4j2,
 - écriture de tests unitaires avec JUnit5.

Sommaire

PARTIE I : ANALYSE DU ROBOT DE CUISINE.....	2
PARTIE II : ET SI ON AJOUTAIT UNE NOUVELLE FONCTIONNALITE ?.....	3
PARTIE III : ANALYSE DE LA QUALITE DU CODE	3
PARTIE IV : MISE EN PLACE D'UN LOGGER.....	5
PARTIE V : TESTS ET COUVERTURE DU CODE	6
PARTIE VI : <i>REFACTORING</i> DU CODE	7



Dans les archives de l'école, nous avons retrouvé le code d'un robot de cuisine. Ce code est donné dans la Figure 1. Après une analyse du code, vous rendrez le code du robot fonctionnel et livrable, puis vous lui ajouterez une nouvelle fonctionnalité.

Partie I : Analyse du robot de cuisine

```
1 package fr.esisar.cookbot;
2
3 public class CookBot {
4
5     static final int ON = 0;
6     static final int OFF = 1;
7     static final int COOK = 2;
8
9     private int state;
10
11     public CookBot() {
12         super();
13         state = ON;
14     }
15
16     public void switchOn() {
17         if (state == ON) {
18             System.out.println("CookBot is already switched on");
19         } else if (state == COOK) {
20             state = ON;
21             System.out.println("CookBot is switched on...");
22         } else if (state == OFF) {
23             state = ON;
24             System.out.println("CookBot is switched on...");
25         }
26     }
27
28     public void regularCook() {
29         if (state == ON) {
30             state = COOK;
31             System.out.println("CookBot is cooking...");
32         } else if (state == COOK) {
33             System.out.println("CookBot is already cooking...");
34         } else if (state == OFF) {
35             System.out.println("CookBot cannot cook when it is switched off");
36         }
37     }
38
39     public void switchOff() {
40         if (state == ON) {
41             state = OFF;
42             System.out.println("CookBot is switched off");
43         } else if (state == COOK) {
44             System.out.println("CookBot cannot be switched off while cooking");
45         } else if (state == OFF) {
46             System.out.println("CookBot is already switched off");
47         }
48     }
49
50     public int getState() {
51         return state;
52     }
53 }
```

Figure 1 : Code du robot de cuisine.

Question 1 : Réalisez, avec le logiciel StarUML, le diagramme de classes du robot de cuisine (Figure 1).

Question 2 : Réalisez, avec le logiciel StarUML, le diagramme d'états-transitions du robot de cuisine.

Indice : Dans StarUML, lorsque vous cliquez sur un élément, vous retrouvez, en bas à droite de votre écran, un éditeur qui contient l'ensemble des caractéristiques de l'élément cliqué que vous pouvez modifier.

Partie II : Et si on ajoutait une nouvelle fonctionnalité ?

Le robot de cuisine est un excellent assistant culinaire, idéal pour préparer une grande variété de plats. À l'avenir, il pourrait être encore plus performant — et sans doute plus coûteux — s'il intégrait la possibilité de mijoter de délicieux petits plats à feu doux.

Question 3 : Complétez le diagramme d'états-transitions, puis le diagramme de classes réalisés précédemment pour ajouter un nouveau mode de cuisson SLOW_COOK, qui permette de faire cuire doucement les ingrédients. Lorsque le robot est dans l'état SLOW_COOK et que l'on souhaite :

- l'allumer (switchOn), il passe dans l'état ON (*CookBot is switched on...*),
- l'éteindre (switchOff), il reste dans l'état SLOW_COOK (*CookBot cannot be switched off while slowly cooking*),
- passer à une cuisson classique (regularCook), il passe dans l'état COOK (*CookBot is cooking...*).

De plus, la fonction slowCook sera également ajouté au robot :

- si le robot est dans l'état ON, il passera dans l'état SLOW_COOK (*CookBot is cooking at a lower temperature...*),
- si le robot est dans l'état OFF, il restera arrêté (*CookBot cannot cook when it is switched off*),
- si le robot est dans l'état COOK, il passera dans l'état SLOW_COOK (*CookBot is cooking at a lower temperature...*),
- si le robot est dans l'état SLOW_COOK, il restera dans cet état (*CookBot is already cooking at a lower temperature...*).

Question 4 : Complétez le code du robot pour prendre en considération la nouvelle fonctionnalité de cuisson douce et vérifiez qu'il est fonctionnel.

Question 5 : Analysez la qualité du code du robot de cuisine autant d'un point de vue modélisation qu'implémentation. Votre réponse doit être argumentée.

Partie III : Analyse de la qualité du code

Question 6 : Analysez la qualité du projet avec le logiciel SonarQube¹². Quels sont les résultats de l'analyse faite par SonarQube ? Concordent-ils avec votre analyse ?

¹ <https://www.sonarsource.com/products/sonarqube/>

² « SonarQube est un logiciel libre, qui aide à la détection, la classification et la résolution de défauts dans le code source. Il permet d'identifier les duplications de code, de mesurer le niveau de documentation et de connaître la couverture de tests déployée. » (<https://fr.wikipedia.org/wiki/SonarQube>).

Configuration de SonarQube :

1. Se connecter avec ses identifiants Agalan au serveur SonarQube de l'école à l'adresse suivante : <http://sonar.esisar.grenoble-inp.fr:9000>
2. Cliquer sur « Create project », puis choisir « Manually »
3. Remplir le formulaire en choisissant comme nom et clé du projet : « CookBot-XXX » avec XXX votre nom.

Create a project

Project display name *

 
Up to 255 characters. Some scanners might override the value you provide.

Project key *

 
The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

The name of your project's default branch [Learn More](#)

Next

4. Puis « Next », cocher « Use the global setting », puis « Create project ».
5. Choisir « Locally » pour le répertoire à analyser, puis « Generate a project token » qui générera un jeton valide 30 jours. Cliquer sur « Generate ». Sauvegarder le jeton généré, puis « Continue ».

Analysis Method > Locally

Analyze your project

We initialized your project on SonarQube, now it's up to you to launch analyses!

1 Provide a token

Analyze "CookBot": `sqp_cd0d2c936eafa793ce4b0244dc8a84799ee7ef6d` 

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#).

Continue

Configuration de votre projet :

1. Compléter le fichier pom.xml :
 - a. en ajoutant le plugin sonar-maven-plugin dans les plugins :

```
<plugin>
  <groupId>org.sonarsource.scanner.maven</groupId>
  <artifactId>sonar-maven-plugin</artifactId>
  <version>5.1.0.4751</version>
</plugin>
```

- b. en définissant les propriétés suivantes :

Nom de la propriété	Valeur de la propriété
sonar.projectKey	La clé du projet SonarQube
sonar.projectName	Le nom du projet SonarQube
sonar.host.url	L'adresse du serveur SonarQube avec son numéro de port

2. Exécuter la commande :

```
mvn clean install sonar:sonar -Dsonar.token="sqp_XXX"
```

où sqp_XXX est le jeton qui a été généré précédemment.

Partie IV : Mise en place d'un logger

Question 7 : Est-ce qu'en ajoutant un logger la qualité du code s'améliore ? Argumentez et illustrez votre réponse.

Configuration du logger pour Apache Maven :

Pour utiliser le logger Apache Log4j2, vous devez ajouter dans le fichier pom.xml de nouvelles dépendances, que vous retrouvez dans le dépôt central Maven³ :

groupId	artifactId	version
org.apache.logging.log4j	log4j-core	2.24.3
org.apache.logging.log4j	log4j-api	2.24.3

Utilisation du logger :

Modifiez le code de la classe CookBot pour mettre en place le logger en ajoutant l'attribut suivant :

```
private static final Logger LOGGER = LogManager.getLogger(CookBot.class);
```

Le type Logger et la classe LogManager sont définis dans le package org.apache.logging.log4j.

Remplacez tous les System.out.println par l'appel à une méthode telle que info(), warn() ou autres du logger. Le contenu des messages doit être inchangé.

Configuration du logger pour le projet :

Pour que votre code fonctionne correctement, vous devez ajouter un fichier de configuration nommé log4j2.xml dans le répertoire src/main/resources. Ce fichier active l'affichage dans la console de tous les messages ayant un niveau supérieur ou égal au niveau trace.

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration>
  <Appenders>
    <Console name="Console">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="trace">
      <AppenderRef ref="Console"/>
    </Root>
  </Loggers>
</Configuration>
```

Vérifiez que votre code est toujours fonctionnel avant d'évaluer la qualité du code avec SonarQube.

³ <https://mvnrepository.com/>

Partie V : Tests et couverture du code

Pour améliorer la qualité du code, vous allez ajouter des tests à votre projet.

Configuration des tests avec JUnit5 pour Apache Maven :

Pour utiliser Junit5, vous devez ajouter dans le fichier pom.xml de nouvelles dépendances, que vous retrouvez dans le dépôt central Maven⁴ :

groupId	artifactId	version	scope
org.junit.jupiter	junit-jupiter-api	5.12.2	test
org.junit.jupiter	junit-jupiter-engine	5.12.2	test
org.junit.platform	junit-platform-launcher	1.12.2	test

Développement d'une classe de tests :

Dans le répertoire src/test/java, puis dans le package fr.esisar.cookbot, ajoutez la classe suivante et complétez-la :

```
package fr.esisar.cookbot;

class CookStateTest {

    private CookBot cookBot;

    @BeforeEach
    void initializeCookBot() {
        cookBot = new CookBot();
        cookBot.regularCook();
        assertEquals(CookBot.COOK, cookBot.getState());
    }

    @Test
    void switchOn() {
        cookBot.switchOn();
        assertEquals(CookBot.ON, cookBot.getState());
    }

    @Test
    void regularCook() {
        // A compléter
    }

    @Test
    void slowCook() {
        // A compléter
    }

    @Test
    void switchOff() {
        // A compléter
    }
}
```

Exécution des tests avec Maven :

Vérifiez que les tests s'exécutent correctement avec la commande : mvn clean test.

Pour que le résultat des tests soient pris en considération par SonarQube, il faut calculer la couverture du code qui se fait avec l'outil Jacoco. Le fichier pom.xml est déjà configuré pour l'utilisation de Jacoco (propriétés, dépendances et plugin). Utilisez la commande suivante pour générer le rapport Jacoco dans l'analyse de SonarQube :

⁴ <https://mvnrepository.com/>

```
mvn clean install sonar:sonar -Dsonar.token="sqp_XXX"
```

Question 8 : Est-ce que la qualité du code s'est améliorée ? Ajoutez des classes de tests supplémentaires (SlowCookStateTest, OnStateTest et OffStateTest) pour améliorer la qualité de votre code. Vous pouvez refuser de fixer la contrainte suivante :



Partie VI : Refactoring du code

Avant de commencer cette partie, déposez obligatoirement sur Chamilo la version du projet réalisé jusqu'à présent.

Le *refactoring* (ou restructuration en français) est le processus de modifier le code source d'un programme sans changer son comportement externe, dans le but de l'améliorer (lisibilité du code, réduction de la complexité, maintenance et évolution facilitées, élimination de duplications, correction d'une dette technique, ajout de futures fonctionnalités facilité...). Le *refactoring* ne permet pas de corriger des bugs, ni d'ajouter des fonctionnalités et il ne doit pas apporter de changement à l'utilisateur.

Dans cette partie, vous allez *refactorer* le code du robot de cuisine en utilisant un patron de conception (en anglais, *design pattern*). L'objectif est de garder les fonctionnalités en améliorant la qualité du code.

Un patron de conception est un arrangement caractéristique de modules/classes, reconnu comme une bonne pratique en réponse à un problème de conception logiciel. Il décrit une solution standard, utilisable dans la conception de différents logiciels. Il existe de nombreux patrons de conception, qui sont issus de l'expérience de développement. L'ensemble des patrons de conception dédiés à la conception sont rassemblés dans l'ouvrage de référence « Design Patterns – Elements of Reusable Object-Oriented Software » rédigé par E. Gamma, R. Helm, R. Johnson et J. Vlissides, surnommés le Gang of Four (GoF). Ci-après vous avez un extrait de ce livre qui décrit le patron de conception State.

State pattern

Intent:

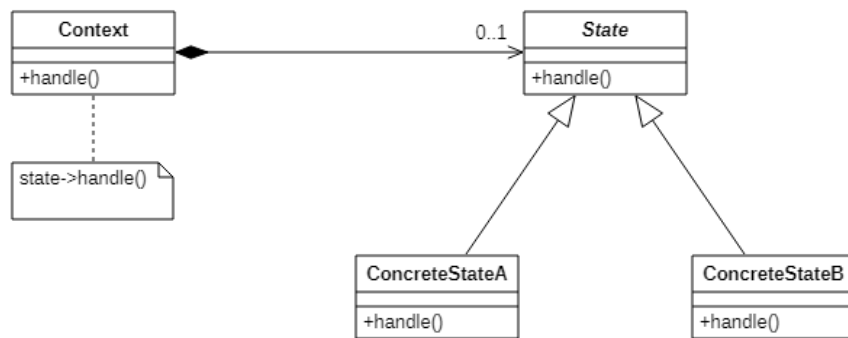
Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Applicability:

Use the State pattern in either of the following cases:

- An object's behavior depends on its state, and it must change its behavior at run-time depending on that state.
- Operations have large, multipart conditional statements that depend on the object's state. This state is usually represented by one or more enumerated constants. Often, several operations will contain this same conditional structure. The State pattern puts each branch of the conditional in a separate class. This lets you treat the object's state as an object in its own right that can vary independently from other objects.

Structure:



Participants:

- Context:
 - Defines the interface of interest to clients.
 - Maintains an instance of a ConcreteState subclass that defines the current state.
- State:
 - Defines an interface for encapsulating the behavior associated with a particular state of the Context.
- ConcreteState subclasses:
 - Each subclass implements a behavior associated with a state of the Context.

Question 9 : Expliquez quel est l'intérêt du patron de conception State et pourquoi peut-il vous permettre d'améliorer le code du robot de cuisine réalisé précédemment.

Dans les archives de l'école, en plus du code donné précédemment, nous avons trouvé une ébauche de modélisation, donnée dans la Figure 2, qui semble respecter le patron de conception State.

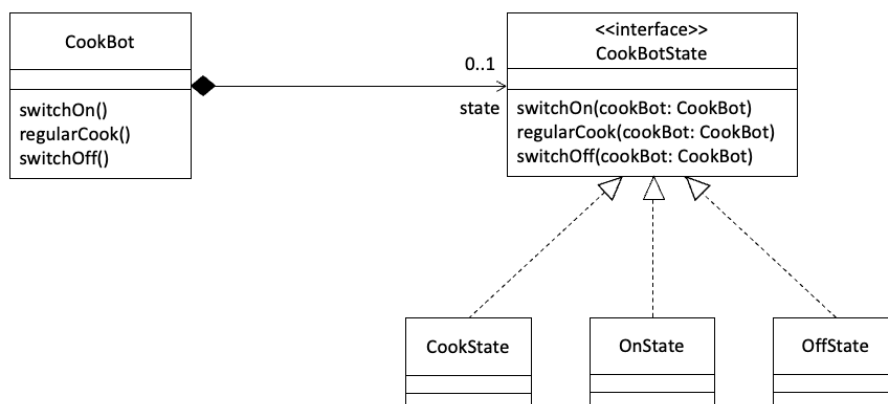


Figure 2 : Ebauche de diagramme de classes.

Question 10 : Est-ce que le diagramme de classes donné dans la Figure 2 respecte le modèle général donné dans le patron de conception ? Pour vous aider, pourquoi trouve-t-on dans le diagramme de

classes une interface à la place d'une classe abstraite ? Quels sont les arguments qui permettent de faire ce choix d'implémentation (interface vs. classe abstraite) ?

En plus de l'ébauche du diagramme de classes, nous avons retrouvé un morceau de code de la classe CookBot, qui ne semble pas complète.

```
public class CookBot {  
    private CookBotState state;  
  
    public CookBot() {  
        super();  
        this.state = new OnState();  
    }  
  
    public void regularCook() {  
        state.regularCook(this);  
    }  
  
    public CookBotState getState() {  
        return state;  
    }  
  
    /**  
     * Set the current state. Normally only called by classes implementing  
     * the CookBotState interface.  
     *  
     * @param state the new state of this context (CookBot)  
     */  
    public void setState(CookBotState state) {  
        this.state = state;  
    }  
}
```

Question 11 : Le diagramme de classes donné dans la Figure 2 est une ébauche d'un diagramme de conception. Il manque de nombreux éléments pour qu'il devienne un diagramme de classes pour l'implémentation. Complétez le diagramme de classes pour qu'il contienne l'ensemble des éléments nécessaires à l'implémentation.

Question 12 : Implantez le code du robot de cuisine en respectant le diagramme de classes que vous avez réalisé précédemment et qui doit respecter le patron de conception State.

Votre code doit également respecter toutes les bonnes pratiques vues en cours de Génie Logiciel.

- Vous devez utiliser le logger Apache Log4j2 afin d'avoir des messages cohérents et précis, indiquant précisément leur provenance (c'est-à-dire la classe émettrice).
- Vous devez reprendre les tests JUnit5 réalisés précédemment en les adaptant à votre nouveau code. Pour les tests portant sur l'état des objets, vous utiliserez désormais la méthode `assertInstanceOf()`⁵ à la place de `assertEquals()`.
- Vous devez analyser la qualité du nouveau code. Quels résultats obtenez-vous ? Analysez les résultats obtenus.

5

[https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org.junit.jupiter.api/Assertions.html#assertInstanceOf\(java.lang.Class,java.lang.Object\)](https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org.junit.jupiter.api/Assertions.html#assertInstanceOf(java.lang.Class,java.lang.Object))